

EduGenie: AI-Powered Educational Assistant

Project Documentation

Project Description:

EduGenie is a lightweight AI-powered educational assistant designed to simplify and enhance the learning experience through the power of Generative AI. Developed for students, self-learners, and educators, EduGenie provides intelligent educational support by helping users understand concepts, answer questions, generate quizzes, summarize learning materials, and receive personalized learning recommendations.

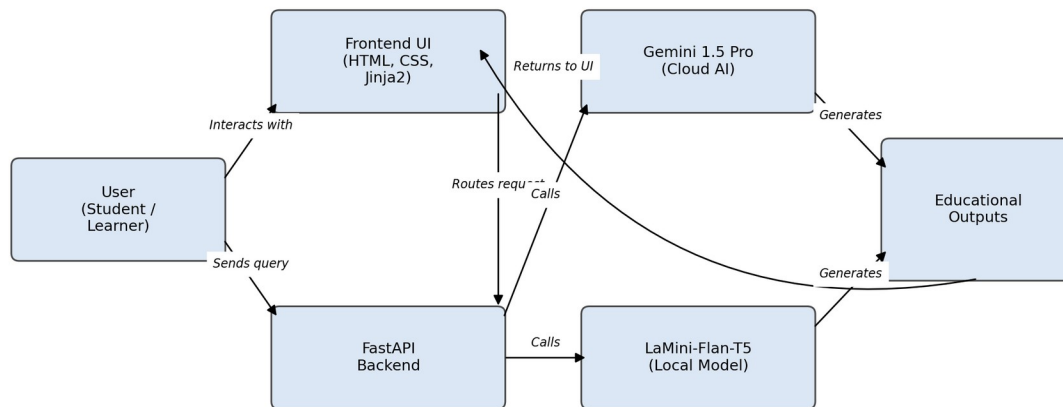
The platform aims to make learning more interactive, accessible, and efficient for users across different academic levels and subject domains. Built using FastAPI for the backend and a responsive HTML + CSS frontend, EduGenie combines the capabilities of lightweight local models and cloud-based AI services to deliver fast and accurate educational assistance. Its optimized architecture allows it to run efficiently on resource-constrained devices such as the Mac M1, making it suitable for both personal learning and educational projects.

Key Features: Intelligent Question Answering, Simplified Concept Explanations, AI-Powered Quiz Generation, Educational Text Summarization, Personalized Learning Path Recommendations, and an Interactive, User-Friendly Interface.

Scenarios

- **Scenario 1:** A student wants to learn about oceans and rivers. They ask, "Which is the largest ocean?" EduGenie instantly provides an accurate answer along with additional educational context.
- **Scenario 2:** A student studying the Pythagoras Theorem wants to assess their understanding. They select "Generate Quiz," and EduGenie creates topic-specific questions for self-evaluation.
- **Scenario 3:** A learner interested in SQL requests a structured learning roadmap. EduGenie generates a personalized learning path covering beginner, intermediate, and advanced topics with study recommendations and progression guidance.

Technical Architecture:



Description: EduGenie utilizes a modular architecture that separates the user-facing interface from the AI processing layer. The frontend, built with HTML, CSS, and Jinja2 templates, allows students to submit questions, request explanations, generate quizzes, or ask for a learning roadmap. The FastAPI backend receives these requests, validates the input, and routes it to the appropriate AI engine — Google's Gemini 1.5 Pro for cloud-based reasoning and content generation, or the locally hosted LaMini-Flan-T5 model for lightweight, low-latency tasks. Responses are parsed, formatted, and returned to the frontend for display.

Note: If you are using a database in your project, definitely add an ER Diagram along with a description.

Pre-requisites:

- **Python Programming:** [Python Documentation](#)
- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Generative AI & Prompt Engineering:** [Google AI \(Gemini\) Documentation](#)
- **Hugging Face Transformers:** [Transformers Documentation](#)
- **HTML, CSS & Jinja2 Skills:** [Jinja2 Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Uvicorn Documentation](#)

Project Workflow:

Milestone 1: Environment Setup & Dependency Configuration

- Activity 1.1: Install and configure FastAPI and Uvicorn for the backend server.
- Activity 1.2: Install Transformers and Torch to support the local LaMini-Flan-T5 model.
- Activity 1.3: Install the Google Generative AI SDK for Gemini 1.5 Pro integration.
- Activity 1.4: Set up HTML, CSS, and Jinja2 templating for the frontend.

Milestone 2: AI Model Selection & Integration

- Activity 2.1: Integrate Gemini 1.5 Pro for high-quality reasoning, explanations, and quiz generation.
- Activity 2.2: Set up LaMini-Flan-T5 as a lightweight local model for fast, low-resource tasks.
- Activity 2.3: Design prompt templates for each educational task.
- Activity 2.4: Implement response handling and formatting logic.

Milestone 3: Educational AI Pipeline

- Activity 3.1: Develop the Question Answering pipeline.
- Activity 3.2: Develop the Concept Explanation pipeline.
- Activity 3.3: Develop the Quiz Generation pipeline.
- Activity 3.4: Develop the Text Summarization pipeline.
- Activity 3.5: Develop the Personalized Learning Recommendations pipeline.

Milestone 4: Backend API Development

- Activity 4.1: Build FastAPI routes for question answering and explanations.
- Activity 4.2: Build FastAPI routes for quiz generation and summarization.
- Activity 4.3: Build FastAPI routes for learning recommendations.

Milestone 5: Frontend UI Development

- Activity 5.1: Create the query input form and task selection interface.
- Activity 5.2: Build the quiz generation and summary display components.
- Activity 5.3: Build the learning recommendations display.

Milestone 6: AI Prompt Engineering & Response Handling

- Activity 6.1: Design structured prompts for each educational feature.
- Activity 6.2: Implement structured response parsing and formatting.

Milestone 7: Educational Content Processing

- Activity 7.1: Implement content preprocessing and input validation.
- Activity 7.2: Implement response parsing and formatting for educational content.

Milestone 8: Testing & Local Deployment

- Activity 8.1: Perform integration testing across all AI features.

- Activity 8.2: Implement error handling and run local deployment.

Milestone 1: Environment Setup & Dependency Configuration

In this milestone, the project environment is configured with all core dependencies required to run EduGenie, spanning the FastAPI backend, the AI/ML libraries, and the HTML/CSS/Jinja2 frontend.

Activity 1.1: Install Backend Dependencies

Create a virtual environment and install FastAPI, Uvicorn, and Jinja2:

```
python -m venv venv
venv\Scripts\activate
(venv) pip install fastapi uvicorn jinja2 python-multipart
```

Activity 1.2: Install AI/ML Dependencies

Install Transformers, Torch, and the Google Generative AI SDK:

```
(venv) pip install transformers torch
(venv) pip install google-generativeai
(venv) pip install python-dotenv
```

Activity 1.3: Configure Environment Variables

Create a .env file in the project root to securely store the Gemini API key:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Load the key at application startup using python-dotenv:

```
from dotenv import load_dotenv
import os
import google.generativeai as genai

load_dotenv()
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
```

Activity 1.4: Set Up Application Structure

Organize the project directory to separate routes, models, templates, and static assets:

```
edugenie/
  main.py
  routers/
    qa.py
    quiz.py
    summarize.py
    recommend.py
  models/
    lamini_model.py
    gemini_model.py
  templates/
    index.html
  static/
    css/
    js/
  .env
```

requirements.txt

Milestone 2: AI Model Selection & Integration

This milestone focuses on integrating both the cloud-based Gemini 1.5 Pro model and the lightweight, locally hosted LaMini-Flan-T5 model, so EduGenie can balance response quality with speed and resource efficiency.

Activity 2.1: Gemini 1.5 Pro Integration

```
import google.generativeai as genai

model = genai.GenerativeModel('gemini-1.5-pro')

def ask_gemini(prompt: str) -> str:
    response = model.generate_content(prompt)
    return response.text
```

Activity 2.2: LaMini-Flan-T5 Setup

```
from transformers import pipeline

local_pipeline = pipeline(
    'text2text-generation',
    model='MBZUAI/LaMini-Flan-T5-248M'
)

def ask_local_model(prompt: str) -> str:
    result = local_pipeline(prompt, max_length=256)
    return result[0]['generated_text']
```

Activity 2.3: Prompt Engineering

Structured prompt templates are defined for each educational task, ensuring consistent, well-formatted outputs:

```
PROMPT_TEMPLATES = {
    'qa': 'Answer this question clearly for a student: {query}',
    'explain': 'Explain this concept in simple terms: {query}',
    'quiz': 'Generate 5 quiz questions with answers on: {query}',
    'summarize': 'Summarize the following text for a student: {query}',
    'roadmap': 'Create a beginner-to-advanced learning path for: {query}'
}
```

Activity 2.4: Response Handling

A routing function selects the appropriate model based on task complexity and desired response speed, then formats the returned text for the frontend.

```
def generate_response(task: str, query: str, use_gemini: bool = True) -> str:
    prompt = PROMPT_TEMPLATES[task].format(query=query)
    if use_gemini:
        return ask_gemini(prompt)
    return ask_local_model(prompt)
```

Milestone 3: Educational AI Pipeline

Milestone 3 brings the five core learning features to life as distinct, reusable pipeline functions that sit between the API routes and the AI models.

Activity 3.1: Question Answering

- Accepts a free-form student question and returns a direct, accurate answer with brief educational context.

Activity 3.2: Concept Explanation

- Breaks down complex academic concepts into simplified, easy-to-understand explanations tailored to the student's level.

Activity 3.3: Quiz Generation

- Generates topic-specific multiple-choice or short-answer questions, along with answers, for self-evaluation.

Activity 3.4: Text Summarization

- Condenses long study material, notes, or articles into concise, easy-to-digest summaries.

Activity 3.5: Personalized Learning Recommendations

- Produces a structured learning roadmap covering beginner, intermediate, and advanced topics with progression guidance.

Milestone 4: Backend API Development

Milestone 4 focuses on exposing each educational feature as a FastAPI endpoint, so the frontend can call them via simple HTTP requests.

Activity 4.1: Define Core Routes

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title='EduGenie API')

class QueryRequest(BaseModel):
    task: str
    query: str
    use_gemini: bool = True

@app.post('/api/generate')
def generate(request: QueryRequest):
    result = generate_response(request.task, request.query, request.use_gemini)
    return {'success': True, 'task': request.task, 'result': result}
```

Activity 4.2: Quiz & Summarization Endpoints

```
@app.post('/api/quiz')
def generate_quiz(request: QueryRequest):
    result = generate_response('quiz', request.query, request.use_gemini)
    return {'success': True, 'quiz': result}

@app.post('/api/summarize')
def summarize_text(request: QueryRequest):
    result = generate_response('summarize', request.query, request.use_gemini)
    return {'success': True, 'summary': result}
```

Activity 4.3: Learning Recommendations Endpoint

```
@app.post('/api/roadmap')
def learning_roadmap(request: QueryRequest):
    result = generate_response('roadmap', request.query, request.use_gemini)
    return {'success': True, 'roadmap': result}
```

Milestone 5: Frontend UI Development

Milestone 5 focuses on building a clean, responsive interface so students can interact with every EduGenie feature from a single page.

Activity 5.1: Query Input & Task Selection

- Build a form where users type their question, topic, or text to summarize.
- Add a task selector (Ask a Question, Explain a Concept, Generate Quiz, Summarize Text, Learning Path).
- Add a toggle to choose between Gemini 1.5 Pro and the local LaMini-Flan-T5 model.

Activity 5.2: Quiz & Summary Display

- Render generated quiz questions as an interactive list with expandable answers.
- Display summarized text in a clean, readable card layout.

Activity 5.3: Learning Recommendations Display

- Display the personalized roadmap as a staged list (Beginner, Intermediate, Advanced) with progression guidance.

Milestone 6: AI Prompt Engineering & Response Handling

This milestone refines how prompts are constructed and how AI responses are structured, ensuring consistent, well-formatted educational content across both models.

Activity 6.1: Structured Prompt Design

- Define task-specific instructions embedded in each prompt (tone, format, target academic level).
- Include formatting instructions so quiz and roadmap outputs can be reliably parsed into structured data.

Activity 6.2: Structured Response Parsing

```
import json
import re

def extract_structured(text: str) -> dict:
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        match = re.search(r'\{[\s\S]*\}', text)
        if match:
            return json.loads(match.group(0))
        raise ValueError('Could not parse structured response')
```

Milestone 7: Educational Content Processing

Milestone 7 covers the preprocessing and validation logic that keeps user input and AI output clean and reliable.

Activity 7.1: Content Preprocessing & Validation

- Trim and sanitize user input before it is sent to the AI models.
- Enforce a maximum input length to keep responses fast and within model limits.
- Reject empty or invalid task selections with a clear error message.

Activity 7.2: Parsing & Formatting

- Normalize quiz and roadmap responses into consistent JSON structures for the frontend.
- Strip any stray markdown formatting from model output before rendering.

Milestone 8: Testing & Local Deployment

The final milestone verifies that EduGenie works end-to-end and is ready for local use and demonstration.

Activity 8.1: Integration Testing

- Test each feature (Q&A, explanations, quizzes, summarization, recommendations) with both Gemini and the local model.
- Verify graceful error handling when the Gemini API is unavailable or rate-limited.

Activity 8.2: Local Deployment

Run the application locally using Uvicorn:

```
(venv) uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

Once running, open a browser and navigate to <http://127.0.0.1:8000> to access EduGenie.

Team Members

- V Voruganti Krishna Sanket — Member
- Naresh C | AP24110010223 — Member
- Pavan Varma Uddaraju — Team Lead
- Sai Subash Raja Kosaraju — Member
- Humaira Tasneem Syed — Member

Skills Required

- Python (Programming Language)
- FastAPI
- Generative Artificial Intelligence
- Natural Language Processing (NLP)
- HTML Editor
- Prompt Engineering
- HTML Application
- CSS Animations
- Gemini AI
- AI/ML Inference

Conclusion:

EduGenie is an AI-powered educational assistant built with FastAPI and styled with a responsive HTML and CSS interface that streamlines learning support for students and self-learners. It combines Google's Gemini 1.5 Pro with the lightweight LaMini-Flan-T5 model to deliver intelligent question answering, concept explanations, quiz generation, text summarization, and personalized learning recommendations.

The project, which included model selection, prompt engineering, backend and frontend development, and local deployment, demonstrates how a hybrid cloud-and-local AI architecture can boost accessibility and efficiency for learners across different academic levels. By completing this project, the team built an AI-powered educational assistant, integrated Gemini and LaMini-Flan-T5 models, developed FastAPI-based APIs, and created a responsive, interactive educational interface — positioning EduGenie for future growth into a more comprehensive learning companion.